

NOVANEST ACADEMY

BACKEND DEVELOPMENT INTERVIEW QUESTIONS

Top 50 Technical Questions & Expert Answers

Tailored dynamically based on the NovaNest Spring Boot, JPA, MySQL, and Security codebase.

1. SPRING BOOT & ARCHITECTURE BASICS

Q1: What is the overall architecture of the NovaNest backend?

Answer: NovaNest is built on a layered architecture using Spring Boot. It consists of the Controller layer (exposing REST APIs), Service layer (containing business logic like UserService and OtpService), JPA Repository layer (database access interfaces extending JpaRepository), and the Model layer (database entities like User, Product, Category, Order, CartItem, etc.).

Q2: What is the purpose of the NovaNestBackendApplication class?

Answer: It is the bootstrap class annotated with @SpringBootApplication that starts the Spring Boot context. It also hosts a CommandLineRunner bean (init) that seeds the database with a default administrator user (admin, admin@novanest.com, password: AdminPassword123!) if it doesn't already exist.

Q3: How does Spring Boot auto-configuration work in NovaNest?

Answer: Spring Boot's @SpringBootApplication includes @EnableAutoConfiguration, which dynamically configures components like Tomcat server, Jackson JSON parser, Hibernate, and DataSource based on dependencies in pom.xml (like spring-boot-starter-web and mysql-connector-j) and properties defined in application.properties.

Q4: Explain dependency injection in the project. Give an example.

Answer: Dependency Injection is achieved via constructor injection. For example, in AdminController.java, the repositories (UserRepository, ProductRepository, etc.) and helper classes (ProductImageHelper, JwtService) are injected through the constructor. This is preferred over @Autowired on fields because it facilitates unit testing and ensures immutability.

Q5: How is logging set up and used in the controllers?

Answer: Logging is implemented using SLF4J and Logback (the default in Spring Boot). An instance of Logger is created in controllers (e.g., AdminController) using LoggerFactory.getLogger(AdminController.class) and used to log events such as admin login attempts, authentication successes, and warning messages.

Q6: What is the default port the backend runs on, and how can it be changed?

Answer: The application runs on port 8080 by default. It is configured in src/main/resources/application.properties using the property server.port=8080.

Q7: What is a DTO and why is it used in the authentication flow?

Answer: DTO stands for Data Transfer Object (e.g., LoginRequest, RegisterRequest, AuthResponse). It decouples the API payload from the database entities, allowing the controller to accept and validate only the necessary input fields without exposing internal database structures or entity relationships.

Q8: How does the backend enable Cross-Origin Resource Sharing (CORS) for the React frontend?

Answer: CORS is configured in SecurityConfig.java. It defines a custom CorsConfigurationSource bean allowing credentials, defining allowed headers (Authorization, Content-Type, etc.), allowed HTTP methods (GET, POST, PUT, DELETE), and allowed origins (like http://localhost:5173).

Q9: What is the role of Spring Boot starter parent in pom.xml?

Answer: The spring-boot-starter-parent project acts as a parent POM, providing default configurations for maven plugins, dependency version management (eliminating the need to specify versions for standard starters), and default Java compiler settings.

Q10: How is the application profile managed in NovaNest?

Answer: Currently, the backend runs under the 'default' profile, but active profiles can be set in application.properties using spring.profiles.active to swap database sources or configurations (like dev vs. prod).

2. SECURITY & AUTHENTICATION (JWT & OTP)

Q11: How is security configured in the project?

Answer: Security is configured in SecurityConfig.java using Spring Security. It sets up state-less session management, JWT authentication filters, CORS filters, password hashing, and configures URL access controls (permitting endpoints like /api/auth/** and admin login, while requiring authentication for resource folders and admin operations).

Q12: Explain how JWT (JSON Web Token) authentication works in NovaNest.

Answer: When a user logs in successfully, the backend returns a signed JWT. The frontend stores this token and sends it in the Authorization header as 'Bearer <token>' in subsequent requests. The JwtAuthenticationFilter intercepts requests, extracts and validates the token, and sets the authentication context in Spring's SecurityContextHolder.

Q13: What details does the JWT token contain in this system?

Answer: The JWT token contains the username (as subject), the token creation time, and the token expiration time. It is signed using a HMAC-SHA256 signature secret key specified in application.properties.

Q14: How does JwtAuthenticationFilter handle requests?

Answer: It extends OncePerRequestFilter. It reads the HTTP Authorization header, checks if it starts with 'Bearer ', validates the token using JwtService, loads the corresponding UserDetails, verifies that the token is not revoked, and populates the UsernamePasswordAuthenticationToken to set authentication context.

Q15: What is CustomUserDetailsService and what does it do?

Answer: It implements Spring Security's UserDetailsService. It overrides loadUserByUsername to look up users from UserRepository by username. If found, it returns a custom user object containing authorities based on the user's role (e.g., ROLE_CUSTOMER, ROLE_ADMIN).

Q16: How are user passwords stored in the database?

Answer: Passwords are encrypted before database insertion using BCryptPasswordEncoder (configured as a bean in SecurityConfig.java). BCrypt uses a salt and cryptographic hashing to ensure that passwords cannot be decrypted or compromised.

Q17: How does token revocation/logout work?

Answer: Token details are tracked in JwtTokenRepository. When a user requests logout, the application calls jwtService.revokeToken(token) which marks the token as revoked/invalid in the database, preventing it from being reused.

Q18: What role check constraints exist in the backend?

Answer: The model defines Role (an enum with values CUSTOMER and ADMIN). In AdminController.java, the login method explicitly validates that the authenticated user has Role.ADMIN before issuing an admin token. Other admin endpoints require administrative roles.

Q19: Explain the OTP verification flow in AuthController.java.

Answer: Users request an OTP for operations like registration verification or password reset. The OtpService generates a 6-digit random number, saves it as an OtpToken in OtpTokenRepository linked to the email with an expiry date, and mails it using MailService.

Q20: How is the OTP expiration validated in OtpService?

Answer: The OtpToken entity has an expiryDate column. During verification, OtpService fetches the token by email and code and verifies that the current timestamp (LocalDateTime.now()) is before expiryDate.

Q21: What happens to old/used OTP tokens?

Answer: When an OTP is verified successfully, or when a new OTP is requested, the system invalidates or deletes existing OTP tokens for that email to prevent replay attacks.

Q22: Why does JwtAuthenticationFilter skip token validation for registration or auth login endpoints?

Answer: Because these endpoints are public and permit anonymous access. The filter checks the request URI path; if it equals /api/admin/login or starts with /api/auth/, it directly forwards the request down the filter chain without checking headers.

Q23: How does the application prevent access from expired JWT tokens?

Answer: JwtService checks the expiration claim in the token using Jwts.parser().verifyWith(secretKey).build().parseSignedClaims(token). If the current time is past the expiration time, the token is rejected and standard HTTP 401 Unauthorized is returned.

Q24: What is the purpose of the app.jwt.secret configuration?

Answer: It is a high-entropy string used to sign and verify the integrity of the JWT. If a malicious client modifies any details in the token, the signature validation fails on the backend since the client lacks the secret.

Q25: How does the system handle concurrent tokens for the same user?

Answer: The repository JwtTokenRepository allows multiple active tokens. Upon logout, only the specific token sent in the request header is revoked.

3. JPA, HIBERNATE & DATABASE MAPPINGS

Q26: How are entities mapped in the project? Give examples of annotations used.

Answer: Entities use Jakarta Persistence (JPA) annotations: @Entity (defines database table mapping), @Table (specifies table name), @Id (primary key), @GeneratedValue (auto-increment policy), and @Column (defines specific column constraints like nullable or length).

Q27: Describe the database relationship between Product and Category.

Answer: There is a @ManyToOne relationship. Many products can belong to a single category. The relationship uses @JoinColumn(name = "category_id", nullable = false) in Product class to map the foreign key, and EAGER fetching is enabled to load category details with the product.

Q28: What is the purpose of the @Transient annotation in Java entities? Give examples from Product.java.

Answer: @Transient marks entity fields that should not be persisted in the MySQL database. In Product.java, imageUrl, discount, and rating are marked @Transient because they are computed dynamically or loaded from secondary tables (like product_images) before rendering.

Q29: How does Hibernate handle primary key generation in the entities?

Answer: It uses @GeneratedValue(strategy = GenerationType.IDENTITY). This delegates primary key generation to the MySQL database's auto-increment feature, ensuring safe concurrent key generation.

Q30: What is the configuration spring.jpa.hibernate.ddl-auto=update used for?

Answer: This tells Hibernate to inspect the schema of the database on startup and compare it to the Java entities, altering tables to add missing columns or tables automatically.

Q31: How are cart items mapped to users and products?

Answer: In CartItem.java, there are @ManyToOne associations pointing to User and Product. The combination maps which product and how many units are placed in a user's active shopping cart.

Q32: What is the difference between @ManyToOne and @OneToMany relationships?

Answer: @ManyToOne defines a child-to-parent reference where many child rows share one parent row (e.g. Product -> Category). @OneToMany represents the inverse parent-to-child relationship (e.g. a Category having a collection of Product instances).

Q33: How are images stored for products in this backend?

Answer: Instead of storing binary data in the database, the backend stores image URLs in a separate table mapped by the ProductImage entity. Each ProductImage has a @ManyToOne relationship with Product. A helper service (ProductImageHelper) fetches images and populates the transient imageUrl on the product.

Q34: What is the fetch strategy utilized in Product entity for relationships?

Answer: EAGER fetching (FetchType.EAGER) is used for the category relation. This automatically queries and joins the category details when querying products, avoiding the N+1 select problem when listing products.

Q35: How does Hibernate populate createdAt and updatedAt fields in the tables?

Answer: In Product.java, the columns are marked insertable = false, updatable = false. This configures Spring/Hibernate to let MySQL automatically handle these timestamps (using default triggers like DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP).

Q36: Explain the database relationship mapping for Order and OrderItem.

Answer: A single Order entity maps to a transactional order. Multiple OrderItem instances are linked to an Order using @ManyToOne (in OrderItem class pointing to Order). This tracks the specific items, prices, and quantities for that purchase.

Q37: Why does ProductRepository extend JpaRepository?

Answer: Extending JpaRepository<Product, Integer> provides the class with built-in CRUD capability (like save, findById, findAll, delete), pagination support, and query derivation (finding products by categories or properties).

Q38: How do MySQL queries map to database schema changes in development?

Answer: Scripts like schema-update.sql show database alterations, like adding column configurations (VARCHAR(512), TEXT, BOOLEAN) to categories, users, or products to sync manual changes with JPA updates.

Q39: What is the transaction isolation level or transaction management in place?

Answer: Spring Boot's default transaction manager uses declarative transactions. Classes or methods can be annotated with @Transactional to ensure ACID compliance during compound database writes.

Q40: How do we prevent SQL injection in NovaNest?

Answer: By utilizing JPA repositories which use Hibernate under the hood. Hibernate uses prepared statements and parameterized queries, separating SQL code from user-provided inputs, neutralizing SQL injection vectors.

4. REST API, EXCEPTION HANDLING & VALIDATION

Q41: How does GlobalExceptionHandler intercept controller exceptions?

Answer: The class is annotated with @RestControllerAdvice. It defines methods annotated with @ExceptionHandler(ExceptionType.class) to catch specific errors (like ValidationException) and return a structured JSON error response with custom HTTP status codes.

Q42: How is HTTP request body validation performed in controllers?

Answer: Controllers use the @Valid annotation on request body parameters (e.g. RegisterRequest in AuthController.java). This triggers the JSR-380 validation framework to inspect fields for validation annotations like @NotBlank, @Email, and @Size.

Q43: What is the purpose of the custom ValidationException?

Answer: It is a custom runtime exception designed to represent business validation failures (e.g. category not found, invalid coupon, or login credentials mismatch). When thrown, it is handled by the GlobalExceptionHandler to return HTTP 400 Bad Request.

Q44: How does the backend return a structured JSON response to the client?

Answer: By returning a ResponseEntity<T> object from controller endpoints. Spring Boot uses Jackson library to automatically serialize Java objects (or maps/lists) into JSON payloads and write them to the HTTP response body.

Q45: Describe the payment integration in PaymentController.

Answer: It integrates Razorpay. When a customer initiates payment, the controller initializes a Razorpay client using key credentials and creates an order instance via razorpayClient.orders.create(...). The Razorpay order ID is returned to the client to complete the transaction on the frontend.

Q46: What is the role of the MailService?

Answer: It sends transactional emails. It uses Spring's JavaMailSender configured with Gmail's SMTP servers to send simple emails containing authentication OTPs or password recovery tokens.

Q47: How are HTTP response status codes customized in GlobalExceptionHandler?

Answer: Methods are annotated with `@ResponseStatus` (e.g., `@ResponseStatus(HttpStatus.BAD_REQUEST)`), or they return a `ResponseEntity` where the HTTP status code (like 400, 401, 404, or 409) is set programmatically.

Q48: What is the difference between @RestController and @Controller?

Answer: `@RestController` is a convenience annotation that combines `@Controller` and `@ResponseBody`. It indicates that the class is a controller where every method returns a domain object directly serialized into the HTTP response body rather than resolving a HTML view template.

Q49: How is security handled on the Razorpay key secrets?

Answer: Keys are defined in `application.properties` under `razorpay.key.id` and `razorpay.key.secret`. In production, these properties are resolved dynamically from environmental variables or secure parameter stores.

Q50: How would you debug an issue where a REST endpoint returns a 500 error?

Answer: I would check the Spring Boot application logs to locate the stack trace. The stack trace indicates the exact line of code where the exception occurred. I would also write a unit/integration test or use Postman to reproduce the payload and step through the code using a debugger.
